

Lecția 8

STRUCTURI DINAMICE DE DATE

1. Tipul referință

Am văzut în lecția anterioară că un graf se poate memora sub forma matricei sale de adiacență. Dacă, însă, graful are foarte multe noduri, folosirea matricei de adiacență este ineficientă. De asemenea, dacă vrem să facem un program pentru un concurs real de admitere în liceu, nu vom putea lucra cu vectori, deoarece nu putem declara vectori de lungimi apropiate de situații reale (500 de candidați, de pildă). Astfel, ar trebui să dispunem de o structură nouă de date, un fel de *listă*, care să păstreze mai multe informații înlănțuite între ele, într-o zonă de memorie mai largă.

Sunt, de asemenea, unele cazuri în care avem nevoie de structuri speciale de date, de exemplu, pentru a memora o ierarhie oarecare, ca de pildă un arbore genealogic. Pentru asemenea cazuri și nu doar, structurile de date prezentate până acum, numite *statice*, nu ne ajută prea mult. Toate inconvenientele pot dispărea, însă, odată cu utilizarea structurilor *dinamice* de date.

Variabile statice, variabile dinamice

Variabilele pot fi *statice* sau *dinamice*. Cele **statice** sunt alocate în timpul compilării, în zone bine definite de memorie. Structura, tipul și locul lor din memorie nu se pot modifica în timpul execuției programului. Statice sunt toate variabilele folosite până acum.

Limbajul Pascal oferă posibilitatea alocării memoriei în timpul executării programului, pentru unele variabile, numite *dinamice*, în funcție de necesitate și, de asemenea, există posibilitatea eliberării memoriei ocupate de ele. Variabilele **dinamice** trebuie să aibă tipul bine definit, însă nu se vor declara în secțiunea `var`. De asemenea, accesul la astfel de variabile nu se face direct. Lor li se pune în corespondență un tip **referință**, în mod biunivoc, despre care se spune că referă sau indică ("pointează").

Variabila de tip referință poate conține referiri numai la variabila dinamică care i-a fost pusă în corespondență. Referirea se realizează prin memorarea în variabila de tip referință a adresei unde este stocată variabila dinamică. Corespondența între variabila dinamică și tipul de referință permite cunoașterea structurii variabilei dinamice.

Pentru variabilele de tip referință se va alocă, în timpul compilării, un spațiu de memorie de 4 octeți, care va conține adresa de memorie a variabilei dinamice referite.

Variabilele dinamice se alocă într-o zonă de memorie numită **heap**, diferită de zona fixă a programului, unde se memorează variabilele statice.

O variabilă dinamică referită va ocupa un spațiu de memorie corespunzător tipului ei: 2 octeți pentru `Integer`, 6 octeți pentru `Real`, 10 octeți pentru un șir de 9 caractere (`String[9]`) etc..

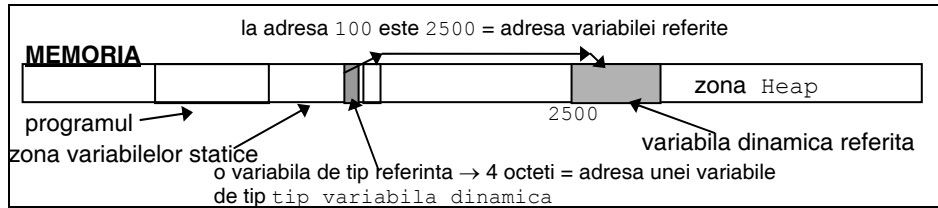
Definirea unui tip referință

Definirea unui tip referință se poate face în secțiunea `type` astfel:

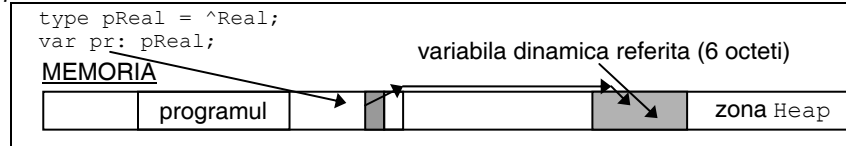
```
type tip_referință = ^tip_variabilă_dinamică;
```

(se citește "pointer la...").

Mulțimea valorilor de tip `tip_referință` constă într-un număr nelimitat de adrese. Fiecare adresă identifică o variabilă de tip `tip_variabilă_dinamică`. La această mulțime de valori se mai adaugă o valoare specială, numită **nil**, care nu identifică nici o variabilă.



Exemplu:

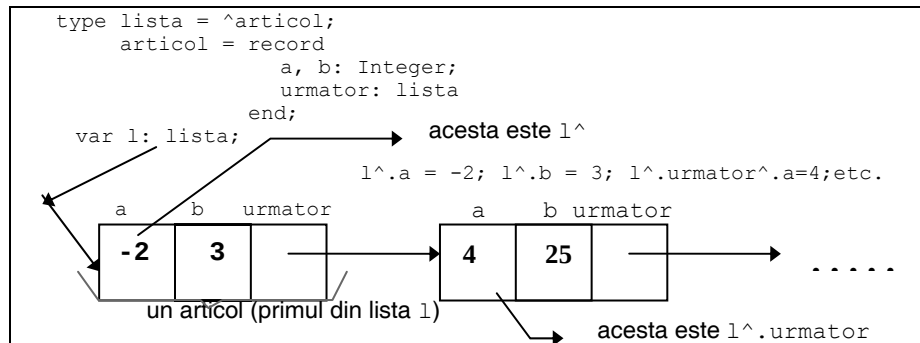


Este permis ca, în momentul întâlnirii tipului variabilei dinamice, acesta să nu fie cunoscut încă (referire înainte); acest tip trebuie declarat mai târziu, în aceeași declarație de tip.

De exemplu:

```
type p_complex = ^complex;
complex = record re, im: Real end;
var r1, r2: p_complex; r3: ^complex; r4: ^Char;
```

O altă facilitate a limbajului constă în posibilitatea utilizării tipurilor care se autoreferă (sunt definite recursiv):



Utilizarea variabilelor dinamice. Avantaje

- ◆ Memorarea unei variabile dinamice se realizează în două faze:
 - alocarea zonei de memorie pentru variabila dinamică, cu procedura **New**;
 - memorarea efectivă, adică depunerea, la adresa pregătită, a datelor corespunzătoare variabilei dinamice.
- ◆ Eliberarea zonei de memorie corespunzătoare unei variabile dinamice, ocupată cu procedura **New**, se realizează cu procedura **Dispose**.

Exemplu:

```
program TestPointer;
type complex = record re, im: Real end; pComplex = ^complex;
var pz: pComplex;
begin
  New(pz); pz^.re:=1.2; pz^.im:=5.6;
  WriteLn('Nr. complex: ', pz^.re, ' ', pz^.im); Dispose(pz); ReadLn
end.
```

Cu variabilele dinamice se pot face toate operațiile care se pot executa cu datele de respectivul tip.

Printre avantajele utilizării pointerilor se numără:

- * folosirea unui spațiu de memorie redus, în cazul utilizării unor structuri complexe de date;
- * folosirea mai eficientă a spațiului de memorie, prin eventuala reutilizare a sa (după Dispose).

Exemplu:

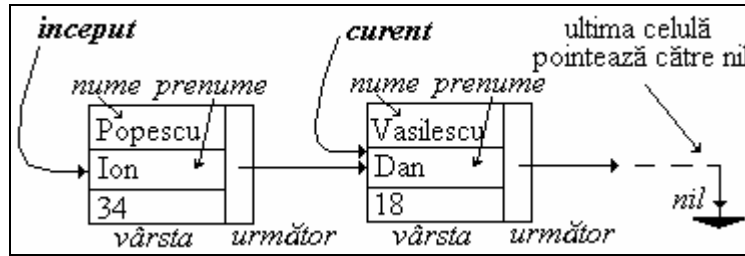
```
type persoana = record
    nume, prenume: String; {2*256 octeti = 512 octeti}
    virsta: Integer {2 octeti, deci 514 octeti, in total}
end;
pPersoana = ^persoana; { 4 octeti }
mult_pers_1 = array[1..50] of persoana;
mul_pers_2 = array[1..50] of pPersoana;
var M1: mult_pers_1; {50*514=25700 octeti}
    M2: mult_pers_2 {50*4=200 octeti, restul e in heap !}
```

Pentru a înlătura dezavantajul dimensiunii foarte mari a vectorilor (50 de elemente), vom realiza o listă înlănțuită, în care fiecare articol (nod) al listei va conține informațiile despre o persoană:

```
type pPersoana = ^persoana;
persoana = record
    nume, prenume: String[20];
    virsta: Integer; urmator: pPersoana
end;
var inceput, curent: pPersoana;
    {elementul de la inceput si cel curent din lista}
```

Exemplu: Crearea unei liste de persoane și căutarea unei persoane în listă.

```
program CuListaDinamica;
type pPersoana = ^persoana;
persoana = record
    nume, prenume: String[20];
    virsta: Integer; urmator: pPersoana
end;
var n,p: String[20]; v: Integer; gasit: Boolean; raspuns: Char;
    inceput, sfirsit: pPersoana;
begin
    inceput:=Nil;
    repeat
        Write('Nume='); ReadLn(n);
        Write('Prenume='); ReadLn(p); Write('Virsta='); ReadLn(v);
        New(curent);
        with curent^ do begin nume:=n; prenume:=p; virsta:=v end;
        inceput:=curent;
        Write('Continuati [D/N] ? '); ReadLn(raspuns)
    until raspuns in ['n','N'];
    Write('Nume cautat = '); ReadLn(n);
    curent:=inceput; gasit:=False;
    while (curent<>Nil) and (not gasit) do
        if curent^.nume=n then gasit:=True else curent:=curent^.urmator;
    if gasit then
        WriteLn('Prenume=',curent^.prenume,', virsta=',curent^.virsta)
    else WriteLn('Persoana inexistentă in lista...'); ReadLn
end.
```



2. Stive și cozi

Listele, implementate dinamic, sunt foarte utile atunci când se lucrează cu multe informații, pe care vectorii se dovedesc incapabili a le stoca, sau ineficienți.

Un caz particular de liste (simplu înălțuite) îl constituie *stivele* și *cozile*. Acestea implementează două mecanisme diferite de intrare și ieșire a elementelor din listă. La ambele feluri de liste, un nod al listei este o înregistrare ce conține o informație (*info*), precum și un pointer (indicator) către următorul element al listei (*prec* sau *urm*).

Prezentare generală

- **Stiva** este o structură dinamică de date reprezentată de o listă simplu înălțuită în care mecanismul de intrare - ieșire a elementelor este de tip **LIFO** - ultimul intrat este primul ieșit (*last in - first out*). Astfel, vom memora o stivă în felul următor:

```

type Stiva = ^Celula;
Celula = record
    info: Integer; { informatia }
    prec: Stiva { precedentul element din stiva }
end;
var S: Stiva;
    
```

Așadar, este de ajuns un pointer către primul element al stivei, pentru a realiza atât operația de adăugare a unui element (numită **Push**), cât și cea de eliminare (**Pop**), deoarece ambele operații se realizează prin partea superioară a listei.

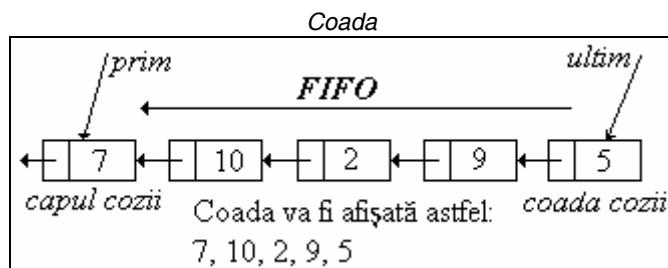
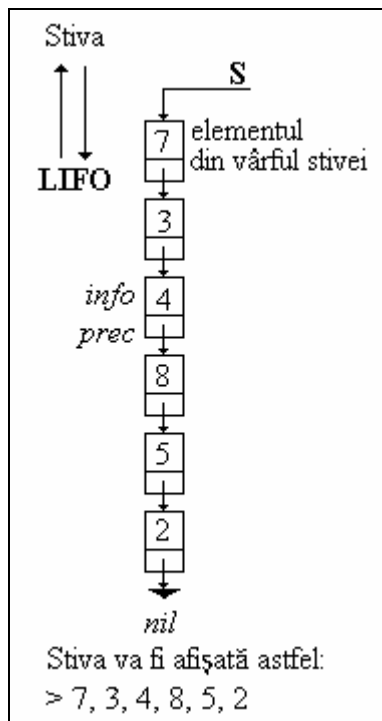
- **Coadă** este o structură dinamică de date reprezentată de o listă simplu înălțuită în care mecanismul de intrare - ieșire a elementelor este de tip **FIFO** - primul intrat este primul ieșit (*first in - first out*). Astfel, vom memora o coadă în felul următor:

```

type PCelula = ^ Celula;
Celula = record
    info: Integer; { informatia }
    urm: PCelula { urmatorul la coada }
end;
Coadă = record prim, ultim: PCelula end;
var C: Coadă;
    
```

Așadar, în cazul cozii, avem nevoie de doi pointeri, unul către primul element al cozii (*capul cozii*), iar altul către ultimul său element (*coada cozii*), deoarece introducerea în listă se face prin spate, iar eliminarea prin față. (Există și o variantă de coadă circulară, în care elementele sunt legate într-un cerc, iar cei doi pointeri, indicând capul și coada cozii, sunt undeva pe acest cerc.).

Stiva



Programe demonstrative

Prezentăm două programe demonstrative care utilizează aceste structuri de date. Elementele din listă sunt numere întregi. Introducerea unui element se face prin simpla scriere a sa, dar trebuie ca elementul să nu fie nici 0, nici -1. Eliminarea unui element (conform mecanismului implementat (LIFO sau FIFO) se realizează prin introducerea lui -1. Oprirea programului se face introducând 0.

1. Programul cu stive. Prin apelul procedurii `Init(S)` se inițializează o stivă `S`, prin `Push(S, elem)` se adaugă elementul întreg `elem`, iar prin `Pop(S, elem)` se scoate `elem` din stiva `S`.

```
program CuStiva;
type Stiva = ^Celula;
```

```
Celula = record
    info: Integer;
```

```

        prec: Stiva
    end;
procedure Init(var S: Stiva);
begin S := Nil end;
procedure Push(var S: Stiva; elem: Integer); { pune }
var C: Stiva;
begin
    New(C); C^.info := elem;
    C^.prec := S; S := C
end;
procedure Pop(var S: Stiva; var elem: Integer); { scoate }
var C: Stiva;
begin
    if S = Nil then
        begin
            Write('Stiva goala ...');
            elem := -1
        end
    else
        begin
            C:=S; elem:=C^.info;
            S:=C^.prec; Dispose(C)
        end
    end;
end;
procedure Afis(S: Stiva);
var C: Stiva;
begin
    C := S;
    Write('Stiva este: ', #16);
    while C <> Nil do
        begin
            Write(C^.info, ', ');
            C := C^.prec
        end;
    WriteLn
end;
var S: Stiva; elem: Integer;
begin
    Init(S); Afis(S);
    repeat
        Write('Dati elementul (-1 = scoate, 0 = stop): ');
        ReadLn(elem);
        if elem <> 0 then
            if elem <> -1 then
                begin
                    Push(S,elem);
                    Afis(S)
                end
            else
                begin
                    Pop(S,elem);
                    WriteLn('Am scos ', elem);
                    Afis(S)
                end
            end
        until elem = 0;
end.

```

2. Programul cu coada. Acesta funcționează la fel ca și cel anterior, doar că procedurile de adăugare și eliminare se numesc Pune, respectiv Scoate, iar mecanismul este FIFO.

```

program CuCoada;
type PCelula = ^Celula;
  Celula = record
    info: Integer;
    urm: PCelula
  end;
  Coada = record
    prim, ultim: PCelula
  end;

procedure Init(var C: Coada);
begin
  C.prim:=Nil; C.ultim:=Nil
end;

procedure Pune(var C: Coada; elem:
Integer);
var P: PCelula;
begin
  New(P); P^.info:=elem;
  P^.urm:=Nil;
  if C.prim=nil then
    {nici un element}
    begin
      C.prim:=P; C.ultim:=P
    end
  else
    begin
      C.ultim^.urm:=P; C.ultim:=P
    end
  end;
end;

procedure Scoate(var C: Coada; var
elem: Integer);
var P: PCelula;
begin
  P := C.prim;
  if C.prim = Nil then
    begin
      Write('Coada este vida ...');
      elem := -1
    end
  else
    begin
      elem := C.prim^.info;
      C.prim := C.prim^.urm;
      Dispose(P)
    end
  end;
end;

procedure Afis(C: Coada);
var P: PCelula;
begin
  Write('Coada este: ');
  P := C.prim;
  while P <> Nil do
    begin
      Write(P^.info, ', ');
      P := P^.urm
    end
  end;
  WriteLn
end;

var C: Coada; elem: Integer;
begin
  Init(C);
  Afis(C);
  repeat
    Write('Dati elementul (-1 =
scoate, 0 = stop): ');
    ReadLn(elem);
    if elem <> 0 then
      if elem <> -1 then
        begin
          Pune(C, elem);
          Afis(C)
        end
      else
        begin
          Scoate(C, elem);
          WriteLn('Am scos ',
elem);
          Afis(C)
        end
      end
    until elem = 0;
end.

```

3. Liste dublu înălțuite

În cazul listelor dublu înălțuite avem, spre deosebire de stive și cozi, următoarele noi elemente:

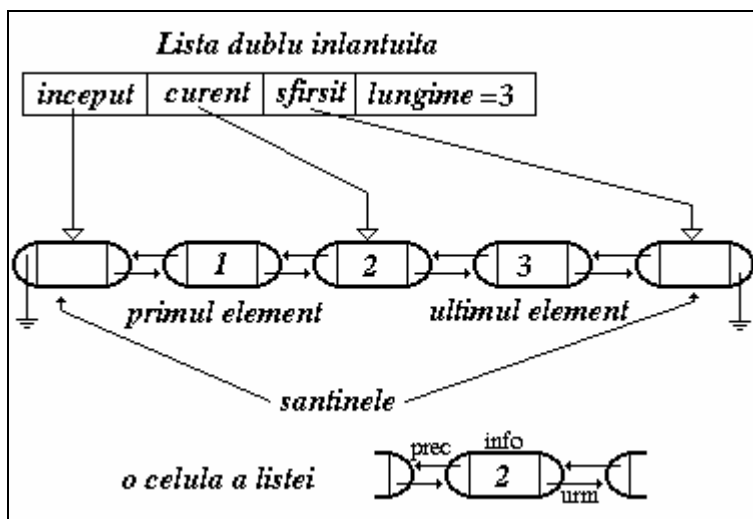
- există doi pointeri speciali: *inceput* și *sfirsit* care point-ează către două celule extreme ale listei, dar care *nu* fac parte din listă; ei se numesc *santinele*;
- există un pointer numit *curent* care indică întotdeauna elementul curent din listă;
- fiecare element a listei este legată prin doi pointeri (*prec* și *urm*) de elementele dinaintea și de după el din cadrul listei;
- avem un câmp *lungime*, care va indica lungimea listei.

Astfel, lista de numere 1, 2, 3, 4, 5 va fi memorată ca în figura de mai jos.

Operațiile ce se cer a se efectua cu o astfel de listă sunt:

- inițializarea listei;
- adăugarea unui element la sfârșitul listei;
- inserarea unui element înaintea elementului curent din listă;
- ștergerea elementului curent din listă.
- afișarea listei.

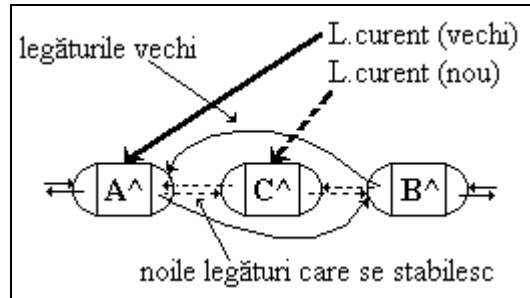
Când se adaugă sau se inserează un nou element în listă, acel element devine cel curent. Când elementul curent se șterge din listă, locul său este preluat de elementul care îl succede, în cadrul listei.



În momentul în care se introduce sau se elimină un element din listă, dispar unele legături și apar altele. De pildă, să presupunem că avem o listă L și, înaintea elementului curent se inserează o informație $elem$. Atunci vom crea o nouă celulă, cu numele C , în care vom pune această informație. Să numim celula curentă A , iar cea de după ea B . Atunci, va trebui să rupem legăturile dintre A și B , stabilind, de asemenea, legături între A și C și între C și B . Totodată, lungimea listei va crește cu o unitate. Aceste lucruri sunt evidențiate în procedura de mai jos și în figura asociată.

```

procedure Insereaza(var L: Lista; elem: tip_info);
var A,B,C: PCelula;
begin
    New(C); A:=L.curent; B:=L.curent^.urm; C^.info:=elem; C^.prec:=A;
    C^.urm:=B; A^.urm:=C; B^.prec:=C; L.lung:=L.lung+1; L.curent:=C
end;
  
```

Procese aproape inverse se realizează la eliminarea elementului curent din listă. Firește, acest lucru se poate realiza doar dacă lista nu este vidă. Pentru a elibera efectiv zona de memorie ocupată de elementul eliminat, se apelează procedura *Dispose*. Eliminarea elementului curent va presupune legarea elementului ce-l precede cu cel ce îl succede.

```

procedure Sterge(var L: Lista);
var C: PCelula;
begin
  with L do
    if lung>0 then
      begin
        C:=curent; C^.prec^.urm:=C^.urm;
        C^.urm^.prec:=C^.prec; curent:=C^.urm; Dispose(C);
        lung:=lung-1
      end
    end;
end;

```

Succesorul elementului eliminat devine element curent. De asemenea, lungimea listei scade cu o unitate.

În continuare prezentăm o nouă variantă a programului pentru admiterea în liceu. De această dată, candidații sunt puși într-o listă dinamică, dublu înlanțuită. Astfel, numărul de candidați va fi limitat doar de memoria rămasă disponibilă în calculator, deci poate fi foarte mare, comparativ cu cazul memorării elevilor într-un vector.

Pentru a realiza ordonarea elevilor după un anumit criteriu (alfabetic sau după medii, descrescător), am implementat o variantă a algoritmului de sortare "*bubble-sort*", care folosește o listă *L* în locul unui vector. În esență, algoritmul este același. Deosebiri apar în parcurgerea listei, precum și în cazul elementelor care se compară: în loc de $x[i]$ și $x[i+1]$, acum se compară elementele *L.curent^.info* și *L.curent^.urm^.info*, de fapt acele câmpuri ale lor care sunt cheie în ordonarea efectuată. În program, secvența care implementează algoritmul "*bubble-sort*" este încadrată.

```

{$X+}
program CuListe;
uses Crt;
type elev = record nume: String; nota1, nota2, media: Real end;
  tip_info=elev;
  PCelula = ^Celula;
  Celula = record info: tip_info; prec, urm: PCelula end;
  Lista = record inceput, curent, sfirsit: PCelula; lung: Integer end;

procedure Init(var L: Lista);
begin
  with L do
    begin
      lung:=0; inceput:=nil; sfirsit:=nil; inceput^.urm:=sfirsit;
      inceput^.prec:=nil; sfirsit^.prec:=inceput;
      sfirsit^.urm:=nil; curent:=inceput
    end
  end;
end;

```

```

        end;
    end;
procedure PozitioneazaLaInceput(var L: Lista);
begin
    while not (L.curent=L.inceput^.urm) do L.curent:=L.curent^.prec
end;
procedure Adauga(var L: Lista; elem: tip_info);
var C: PCelula;
begin
    New(C);
    with C^ do
        begin
            info:=elem; prec:=L.sfirsit^.prec; urm:=L.sfirsit^.prec^.urm
            { sau L.sfirsit}
        end;
    with L do
        begin
            lung:=lung+1; sfirsit^.prec^.urm:=C;
            sfirsit^.prec:=C; curent:=C
        end
    end;
end;
procedure Sterge(var L: Lista);
var C: PCelula;
begin
    with L do
        if lung>0 then
            begin
                C:=curent; C^.prec^.urm:=C^.urm; C^.urm^.prec:=C^.prec;
                curent:=C^.urm; Dispose(C); lung:=lung-1
            end
        end;
end;
procedure Insereaza(var L: Lista; elem: tip_info);
var A,B,C: PCelula;
begin
    New(C); A:=L.curent; B:=L.curent^.urm; C^.info:=elem; C^.prec:=A;
    C^.urm:=B; A^.urm:=C; B^.prec:=C; L.lung:=L.lung+1; L.curent:=C
end;
procedure Afis(L: Lista);
var i: Integer;
begin
    PozitioneazaLaInceput(L); i:=1;
    while not (L.curent = L.sfirsit) do
        begin
            with L.curent^.info do
                WriteLn(i, '. ', nume:20, nota1:6:2, nota2:6:2, media:6:2);
                L.curent:=L.curent^.urm; i:=i+1
            end;
        WriteLn('Total : ', i-1, ' elevi.')
    end;
end;
procedure AdaugareElev(L: Lista);
var E: elev;
begin
    WriteLn('Adaugare elev');
    with E do begin
        Write('Dati numele : '); ReadLn(nume); Write('Dati notele : ');
        ReadLn(nota1, nota2); media:=(nota1+nota2)/2 end;
        Adauga(L, E)
    end;
end;
procedure EliminareElev(var L: Lista);
var numele: String; gasit: Boolean;
begin
    WriteLn('Eliminare elev'); PozitioneazaLaInceput(L);

```

```

WriteLn('Dati numele elevului : '); ReadLn(numele); gasit:=False;
while (not (L.curent = L.sfirst)) and (not gasit) do
  if L.curent^.info.nume = numele then gasit:=True
  else L.curent:=L.curent^.urm;
  if gasit then Sterge(L) else WriteLn('Nu exista acest elev...')
end;
procedure CautareElev(L: Lista);
var numele: String; gasit: Boolean;
begin
  WriteLn('Cautare elev'); PozitioneazaLaInceput(L);
  WriteLn('Dati numele elevului : '); ReadLn(numele); gasit:=False;
  while (not (L.curent = L.sfirst)) and (not gasit) do
    if L.curent^.info.nume = numele then gasit:=True
    else L.curent:=L.curent^.urm;
  if gasit then
    with L.curent^.info do WriteLn(numele, ' ', notal:6:2, nota2:6:2, media:6:2)
  else WriteLn('Elev inexistent...')
end;
procedure ListareEleviDupaNume(L: Lista);
var gata: Boolean; aux: elev;
begin
  WriteLn('Listare elevi dupa nume'); {ordonare elevi dupa nume:bubble-sort}
  repeat
    gata:=True; {ma pozitionez la inceput si parcurg lista}
    PozitioneazaLaInceput(L);
    while not (L.curent^.urm=L.sfirst) do
      begin
        if L.curent^.info.nume > L.curent^.urm^.info.nume then
          begin
            gata := False; aux := L.curent^.info;
            L.curent^.info:=L.curent^.urm^.info; L.curent^.urm^.info:=aux;
          end;
        L.curent:=L.curent^.urm;
      end
    until gata;
  Afis(L) { afisare lista }
end;
procedure ListareEleviDupaMedii(L: Lista);
var gata: Boolean; aux: elev;
begin
  WriteLn('Listare elevi dupa medii');{ordonare elevi:bubble-sort}
  repeat
    gata:=True; {ma pozitionez la inceput, parcurg lista}
    PozitioneazaLaInceput(L);
    while not (L.curent^.urm=L.sfirst)do
      begin
        if L.curent^.info.media < L.curent^.urm^.info.media then
          begin
            gata := False; aux := L.curent^.info;
            L.curent^.info:=L.curent^.urm^.info; L.curent^.urm^.info:=aux;
          end;
        L.curent:=L.curent^.urm;
      end
    until gata;
  Afis(L)
end;
var L: Lista; optiune: Char;
begin
  Init(L);
  repeat
    ClrScr;WriteLn('Meniu:');WriteLn; WriteLn('1 = adaugare elev');
    WriteLn('2 = stergere elev'); WriteLn('3 = cautare elev');

```

```

WriteLn('4 = listare dupa nume'); WriteLn('5 = listare dupa medii');
WriteLn('0 = oprire program'); WriteLn; Write('Optiune=');
ReadLn(optiune); ClrScr;
case optiune of
    '1': AdaugareElev(L); '2': EliminareElev(L);
    '3': CautareElev(L); '4': ListareEleviDupaNume(L);
    '5': ListareEleviDupaMedii(L)
end;
if optiune in ['1'..'5'] then ReadKey
until optiune='0'
end.

```

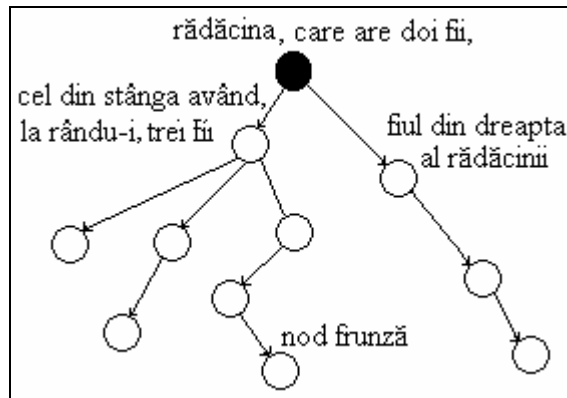
4. Arbori binari

Arborescențe în digrafuri

Fie $H = (V, E)$ un digraf (graf orientat). Se numește **rădăcină** a lui G un vârf $v_0 \in V$, astfel încât oricare ar fi un vârf $v \in V$, există cel puțin un drum de la v_0 la v . Dacă $H = (V, E)$ este un digraf, prin **graful suport** al lui H vom înțelege graful obținut din H , prin renunțarea la orientarea arcelor. H se numește **arborescență** dacă are o rădăcină și graful său suport este arbore.

În informatică, arborescențele sunt numite, prin abuz de limbaj, **arbori**, specificându-se rădăcina și considerând implicită orientarea muchiilor corespunzător parcurgerii drumului unic de la rădăcină la fiecare vârf. Fiecare vârf are astfel, niște *fii*, adică vecinii imediat următori pe drumul de la rădăcină în jos (către *frunze*, adică noduri fără fii). Un arbore cu cel mult doi fii se mai numește și **arbore binar**.

De exemplis:



Privit invers, un arbore seamănă, într-adevăr, cu un arbore din natură (un copac), astfel încât denumirea de *arbore*, ca și cele de *frunză* sau *rădăcină* se justifică. Observăm că putem considera arborile ca fiind organizat pe mai multe *nivele*. Primul nivel este cel al rădăcinii. Urmează nivelul fiilor acesteia ș.a.m.d., până la ultimul nivel, cel al ultimelor frunze. În continuare ne vom ocupa, în mod deosebit, de arborii binari, deoarece, așa cum vom arăta mai târziu, studiul arborilor oarecare se reduce la studiul arborilor binari.

Implementarea arborilor binari

Referirea unui arbore binar și, implicit, definirea sa, va fi făcută printr-un pointer către nodul său rădăcină. Fiecare nod din arbore este o înregistrare cu următoarele elemente:

- o informație `info` de tip întreg;
- doi pointeri către cei doi fii (subarborii stâng și drept) ai nodului: `stg` și `dr`.

În programul care urmează, vom construi astfel de arbori, care au, în plus, următoarea proprietate: fiecare nod din arbore este mai mare sau egal cu nodurile din fiul stâng și mai mic decât nodurile din fiul drept (din punct de vedere al câmpului `info`). Un astfel de arbore se numește **arbore de căutare - sortare**.

O *căutare* a unui element în astfel de arbori este, într-adevăr, ușor de realizat: dacă elementul căutat este identic cu informația din nod, atunci căutarea se încheie cu succes, dacă nu, atunci se pleacă pe una din cele două direcții: dacă elementul căutat este mai mic, atunci se merge pe fiul stâng, altfel pe fiul drept. Dacă se încearcă o trecere dincolo de un nod terminal, deci la `nil`, atunci căutarea eșuează. Procedura din următorul program demonstrativ returnează subarborii având ca rădăcină elementul găsit în arborele în care s-a căutat.

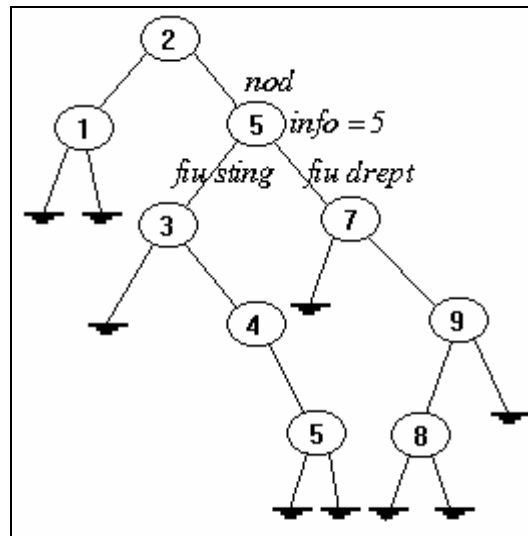
Un arbore binar poate fi parcurs în trei feluri:

- în *inordine*: se parcurge mai întâi, recursiv, în inordine, fiul stâng, apoi rădăcina, apoi fiul drept;
- în *postordine*: se parcurge fiul stâng, apoi cel drept, în final rădăcina;
- în *preordine*: se parcurge rădăcina, fiul stâng și apoi fiul drept.

Se observă că parcurgerea în inordine a unui astfel de arbore duce la afișarea în ordine crescătoare a elementelor din nodurile arborelui, motiv pentru care astfel de arbori pot fi considerați și de *sortare*.

Dacă se inserează, pe rând, elementele următoare în arbore: 2, 5, 7, 3, 4, 9, 1, 5, 8, 0=`stop`, atunci se obține arborele din figura următoare.

Programul care succede figura implementează structura de arbore discutată, precum și principalele operații realizabile cu astfel de structuri de date.



```
program ArboriBinari;
```

```
type arbore = ^nod;
```

```
nod = record info: Integer; stg, dr: arbore end;
```

```
procedure Init(var A: arbore);
```

```
begin A := Nil end;
```

```
{SS-}
```

```
procedure Inserare(var A: arbore; elem: Integer);
```

```
var UnNod: nod;
```

```
begin
```

```
  if A <> Nil then
```

```
    if elem <= A^.info then Inserare(A^.stg, elem) else Inserare(A^.dr, elem)
```

```
  else
```

```
    begin
```

```
      New(A); UnNod.info:=elem; UnNod.stg:=Nil; UnNod.dr:=Nil; A^:=UnNod
```

```
    end
```

```
end;
```

```
procedure Cautare(A: arbore; elem: Integer; var nodul: arbore);
```

```
begin
```

```
  if A = Nil then nodul := Nil
```

```
  else
```

```
    if A^.info = elem then nodul := A
```

```
    else
```

```
      if elem <= A^.info then Cautare(A^.stg, elem, nodul)
```

```
      else Cautare(A^.dr, elem, nodul)
```

```
end;
```

```
procedure PreOrdine(A: arbore);
```

```
begin
```

```
  if A <> Nil then
```

```
    begin Write(A^.info, ', '); PreOrdine(A^.stg); PreOrdine(A^.dr) end
```

```
end;
```

```
procedure PostOrdine(A: arbore);
```

```
begin
```

```
  if A <> Nil then
```

```
    begin
```

```
      PostOrdine(A^.stg); PostOrdine(A^.dr); Write(A^.info, ', ')
```

```
    end
```

```

end;
procedure InOrdine(A: arbore);
begin
    if A <> Nil then
        begin InOrdine(A^.stg); Write(A^.Info, ', '); InOrdine(A^.dr) end
    end;
{$S+}
var A, B: arbore; elem: Integer;
begin
    Init(A);
    repeat
        Write('Dati elementul de inserat (0 = stop): '); ReadLn(elem);
        if elem <> 0 then Inserare(A,elem)
    until elem = 0;
    WriteLn('Arborele in preordine : '); PreOrdine(A); WriteLn;
    WriteLn('Arborele in postordine : '); PostOrdine(A); WriteLn;
    WriteLn('Arborele in inordine', ' => elementele sortate: ');
    InOrdine(A); WriteLn;
    repeat
        Write('Dati elementul cautat'); Write(' (0 = stop): '); ReadLn(elem);
        if elem <> 0 then
            begin
                Cautare(A, elem, B);
                if B <> Nil then
                    begin
                        WriteLn('L-am gasit,', ' arborele in inordine este: ');
                        InOrdine(B); WriteLn
                    end
                else WriteLn('Nu l-am gasit.')
                end
            until elem = 0;
    end.

```

5. Aplicație: derivare formală

Pentru a exemplifica un mod de utilizare a arborilor binari, să reamintim că orice expresie aritmetică poate fi memorată sub forma unui arbore binar. Prin urmare, dacă avem o expresie aritmetică oarecare, putem obține din ea arborele binar asociat ei. Pentru a deriva (formal) respectiva expresie, vom deriva arborele asociat ei, obținând arborele expresiei derivate. Din acest arbore vom construi expresia derivată.

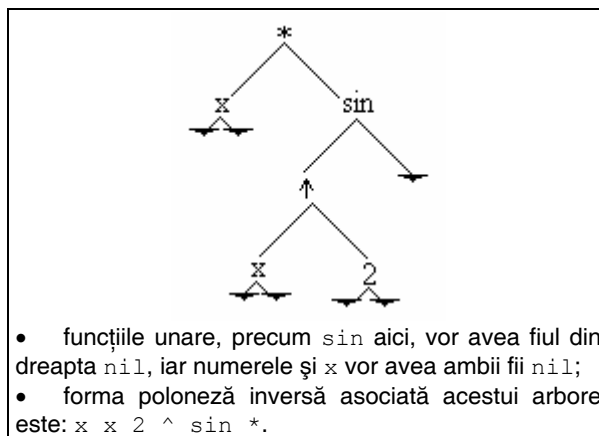
Derivarea formală este o problemă clasică: dându-se o funcție $f(x) = E(x)$, în care $E(x)$ este o expresie în funcție de x , conținând operatorii $+, -, *, /, ^$, precum și funcțiile trigonometrice, logaritmică și exponențială, se cere să se obțină $f'(x) = D(x)$, în care $D(x)$ este expresia obținută din $E(x)$ prin derivare formală, adică aplicând regulile fundamentale de derivare:

$$(f * g)' = f' * g + f * g', \quad (f(g(x)))' = f'(g(x)) * g'(x), \quad (\sin(x))' = \cos(x) \text{ etc..}$$

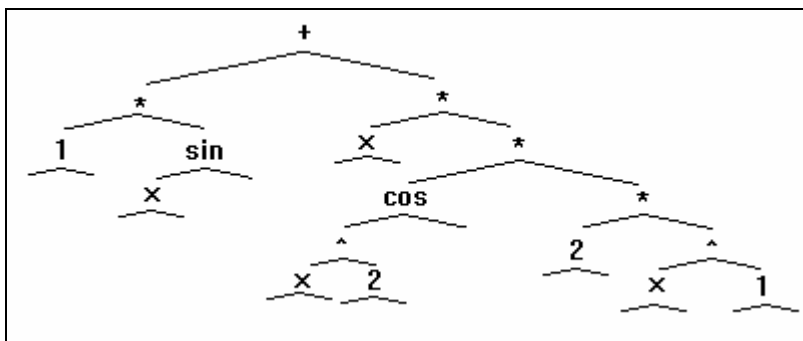
Pentru o expresie algebrică, numim **formă poloneză inversă** notația postfixată a expresiei, adică cea în care mai întâi apar operanzii, apoi operatorul.

Astfel, o expresie va fi reprezentată prin arborele binar corespunzător formei poloneze inverse

Exemplu: pt. expresia $x * \sin(x^2)$ vom avea arborele:



Derivarea acestei expresii ar însemna construirea următorului arbore și apoi scrierea expresiei corespunzătoare, cu operatorii binari în forma infixă, iar funcțiile (unare) în forma prefixă (în figura de mai jos nu am mai figurat `nil`-urile):



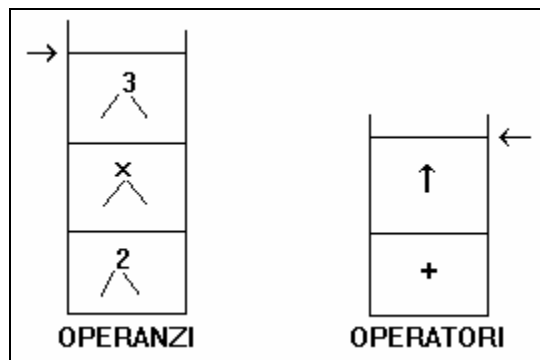
Firește, transformarea dintr-un arbore într-altul s-ar baza pe transformări simple de arbori, care ar reflecta derivările primitive: $f+g \rightarrow f'+g'$, $f*g \rightarrow (f'+g) * (f+g')$ etc..

Vom utiliza o structură arborescentă binară clasică. Cum vom transforma, însă, o expresie obișnuită într-un arbore? Ideea este asemănătoare celei de la evaluatorul de expresii aritmetice: acolo se folosesc două stive, una a operatorilor și funcțiilor - văzute ca operatori unari -, cealaltă a operanzilor, care sunt numere.

Aici vom folosi tot două stive, una a operatorilor și funcțiilor, cealaltă a operanzilor, care, de data aceasta, vor fi arbori mai mici, din care se vor forma arbori mai mari, folosind drept rădăcină ceea ce se afla în vârful stivei cu operații la respectivul moment.

Vom introduce în stiva operațiilor arborele micuț: $/^2 \setminus$ (și nu numărul 2), apoi în stiva operațiilor +, apoi arborele $/^x \setminus$ în stiva operanzilor.

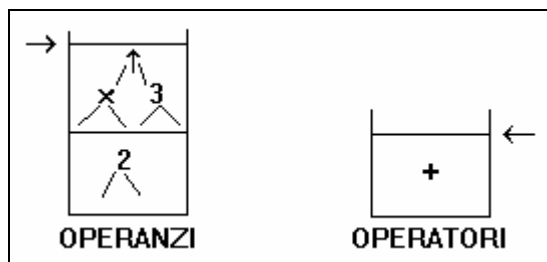
Observăm că $^$ (ridicarea la putere) este de prioritate mai mare ca +, deci nu vom face $2 + x$, ci îl vom introduce și pe $^$ în stiva operațiilor, apoi pe $/^3 \setminus$ în cea a operanzilor.



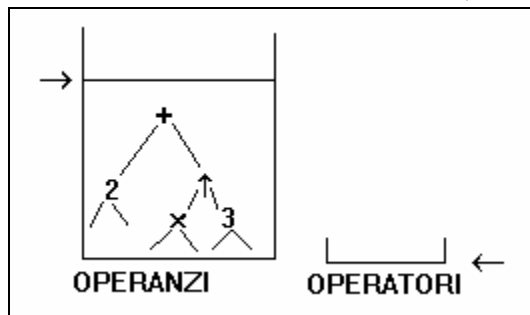
În acest moment va trebui să scoatem cei doi arbori mici din stivă și, împreună cu ^ (scos din stiva operațiilor) drept rădăcină, să formăm un nou arbore:



pe care îl vom introduce în stiva operanzilor:



Singura operație rămasă este + și, procedând ca mai înainte, obținem:



În acest moment, stiva operatorilor este goală, iar stiva operanzilor conține exact arborele ce ne trebuie pentru a putea continua cu derivarea expresiei. Probleme deosebite apar atunci când în vârful stivei operațiilor este - sau /, iar operația succesoare (în expresie) este tot -, respectiv /. (Aceste simboluri reprezintă operații necomutative.)

Programul următor realizează aceste transformări și derivări, însă el poate fi îmbunătățit de către cititor prin adăugarea de operații de simplificare a arborelui rezultat, corespunzător derivării. Aceste operații s-ar baza pe primitivele de genul:

$$f+0 \rightarrow \rightarrow f, \quad f*1 \rightarrow \rightarrow f \quad \text{etc..}$$

Un element de noutate care poate fi observat este prezența unor instrucțiuni grafice, mai ales în cadrul procedurii recursive de afișare a unui arbore binar. Utilizarea graficii are rolul de a face mai plăcută afișarea unui arbore binar; instrucțiunile grafice trebuie luate ca atare, ele nu influențează algoritmul cu nimic. Asupra procedurilor grafice apărute aici se va reveni în lecția 11. Singura observație care trebuie făcută aici este că, pentru a porni programul, trebuie cunoscută calea DOS către fișierul EGAVGA.BGI, din mediul Turbo-Pascal. Pe cuprinsul acestei cărți noi am considerat că aceasta este "C:\BP\BGI". În caz că nu este așa, cititorul va înlocui acest șir de caractere cu cel corect, în procedura de inițializare grafică `OpenGraph` (în apelul procedurii standard `InitGraph`). (De, asemenea, precizăm că `Line(x1,y1,x2,y2)` trasează, pe display-ul grafic, o linie între punctele de coordonate $(x1,y1)$ și $(x2,y2)$, unde coordonatele variază între 0 și 639 (de la stânga la dreapta), respectiv între 0 și 479 (sau 349) de sus în jos. `OutTextXY(x,y,sir)` afișează un șir de caractere în poziția specificată, iar `CloseGraph` închide modul grafic. `ClearViewPort` - în cazul de față - șterge ecranul grafic. Utilizarea subprogramelor grafice se declară prin `uses Graph`, iar în opțiunile de compilare din meniul mediului Turbo-Pascal trebuie să aveți la "Directories" calea către fișierul `GRAPH.TPU`.)

```
program Derivare;
uses Graph,Crt;
const lmax = 3; vmax = 50;
type expresie = string; informatie = string[lmax];
      vector = record
        info:array[1..vmax] of informatie; nr:1..vmax
      end;
      arbore = ^nod;
      nod = record info:informatie; stg,dr:arbore end;
var expresia, expr_fpo, expr_der_fpo: expresie; vectorul: vector;
      arborele, arborele_derivat: arbore;

procedure OpenGraph;
var gd,gm: Integer;
begin
  gd:= Detect; InitGraph(gd,gm,'C:\BP\BGI'); {calea fisierului EGAVGA.BGI}
end;

procedure Vectorizeaza(expr: expresie; var vec: vector);
  procedure Schimba(var x, y: informatie);
    var a: informatie;
    begin a := x; x := y; y := a end;
  var i,j,k: Integer;
  begin
    i := 0; k := 0;
    while i<Length(expr) do
      begin
        Inc(i);
        if expr[i] in ['s','c','l','e'] then
          begin
            Inc(k);
            if expr[i]<>'l' then vec.info[k]:=expr[i]+expr[i+1]+expr[i+2]
            else vec.info[k] := expr[i]+expr[i+1]+' ';
            if expr[i] = 'l' then Inc(i)
            else
              begin vec.info[k] := vec.info[k]+expr[i+2]; Inc(i,2) end
            end
          end
        else
          if expr[i] in ['(',')','+', '-', '*', '/', '^', 'x'] then
            begin Inc(k); vec.info[k] := expr[i] end
          else
            begin { cifre };

```

```

        Inc(k); vec.info[k] := '          '; j := 1;
    while
        not (expr[i] in ['(', ')', '+', '-', '*', '/', '^', 's', 'c', 'l', 'e'])
    do begin vec.info[k][j] := expr[i]; Inc(i); Inc(j) end;
    Dec(i)
end
end;
vec.nr := k
end;
procedure Arborizeaza(vec: vector; var arb: arbore);
const topmax = 25;
var i, top1, top2: Integer;
    operator: array[1..topmax] of String[lmax];
    operand: array[1..topmax] of arbore;
    function prioritate(op:char):byte;
    begin
        case op of
            '(', ')':prioritate := 0; '+', '-':prioritate := 1;
            '*', '/':prioritate := 2; '^':prioritate := 3;
            's', 'c', 'l', 'e':prioritate := 4
        end
    end;
begin { arborizeaza }
    i := 0; top1 := 0; top2 := 1; operator[top2] := '(';
    while (i<=vec.nr) and (top2>0) do
        begin { 1 }
            Inc(i);
            if vec.info[i][1] in ['x', '0'..'9'] then
                begin
                    Inc(top1); New(arb); arb^.info := vec.info[i];
                    arb^.stg := nil; arb^.dr := nil; operand[top1] := arb;
                end
            else
                if vec.info[i][1]='(' then begin Inc(top2);operator[top2]:='(' end
                else
                    begin { 2 }
                        while (top2>0) and (not (operator[top2][1] in ['(', ')'])) and
                            (prioritate(operator[top2][1])>=prioritate(vec.info[i][1]))
                        do begin
                            if operator[top2][1] in ['l', 'c', 's', 'e'] then
                                begin
                                    New(arb); arb^.info := operator[top2];
                                    arb^.stg := operand[top1]; arb^.dr := nil;
                                    operand[top1] := arb
                                end
                            else begin { + - * / ^ }
                                    New(arb); arb^.info := operator[top2];
                                    arb^.stg:=operand[top1-1];arb^.dr:=operand[top1];
                                    operand[top1-1]:=arb; Dec(top1)
                                end;
                            Dec(top2)
                        end; { while - 2 }
                    end
                    if top2>0 then
                        if (operator[top2] <> '(') or (vec.info[i][1] <> ')') then
                            begin Inc(top2); operator[top2] := vec.info[i] end
                        else Dec(top2)
                    end; { else ... }
                end; { while }
            if (i=vec.nr) and (top2=0) then begin New(arb); arb := operand[1] end
            else begin New(arb); arb^.info:='old'; arb^.stg:=nil; arb^.dr:=nil end
        end;
    {$S-}

```

```

procedure Deriveaza(arbi: arbore; var arb: arbore);
var arb1, arb2, arb3, arb4, arb5, arb6, arb7 : arbore;
begin
  New(arb); arb^.info := 'new'; arb^.stg := nil; arb^.dr := nil;
  case arbi^.info[1] of
    '0'..'9': begin arb^.info := '0' end;
    'x': begin arb^.info := '1' end;
    '+', '-': begin
      Deriveaza(arbi^.stg, arb3); arb^.stg := arb3;
      Deriveaza(arbi^.dr, arb4); arb^.dr := arb4;
      arb^.info := '+'
    end;
    '*': begin
      New(arb1); New(arb2); Deriveaza(arbi^.stg, arb3);
      Deriveaza(arbi^.dr, arb4);
      arb^.info := '+'; arb1^.stg := arb3; arb1^.dr := arbi^.dr;
      arb1^.info := '*'; arb2^.stg := arbi^.stg; arb2^.dr := arb4;
      arb2^.info := '*'; arb^.stg := arb1; arb^.dr := arb2;
    end;
    '/': begin
      New(arb1); New(arb2); New(arb5);
      Deriveaza(arbi^.stg, arb3); Deriveaza(arbi^.dr, arb4);
      arb1^.info := '*'; arb1^.stg := arbi^.stg; arb1^.dr := arb4;
      arb2^.info := '*'; arb2^.stg := arb3; arb2^.dr := arbi^.dr;
      arb5^.info := '-'; arb5^.stg := arb1; arb5^.dr := arb2;
      New(arb6); New(arb7);
      arb6^.info := '2'; arb6^.stg := nil; arb6^.dr := nil;
      arb7^.info := '^'; arb7^.stg := arbi^.dr; arb7^.dr := arb6;
      arb^.info := '/'; arb^.stg := arb5; arb^.dr := arb7
    end;
    's': begin
      New(arb1); arb1^.info := 'cos';
      arb1^.stg := arbi^.stg; arb1^.dr := nil; Deriveaza(arbi^.stg, arb2);
      arb^.info := '*'; arb^.stg := arb1; arb^.dr := arb2
    end;
    'c': begin
      Deriveaza(arbi^.stg, arb4); New(arb1); New(arb2); New(arb3);
      arb1^.info := '0'; arb1^.stg := nil; arb1^.dr := nil;
      arb2^.info := 'sin'; arb2^.stg := arbi^.stg; arb2^.dr := nil;
      arb3^.info := '*'; arb3^.stg := arb2; arb3^.dr := arb4;
      arb^.info := '-'; arb^.stg := arb1; arb^.dr := arb3
    end;
    'l': begin
      Deriveaza(arbi^.stg, arb1);
      arb^.info := '/'; arb^.stg := arb1; arb^.dr := arbi^.stg
    end;
    'e': begin
      New(arb2); arb2^.info := arbi^.info;
      arb2^.stg := arbi^.stg; arb2^.dr := arbi^.dr;
      Deriveaza(arbi^.stg, arb1);
      arb^.info := '*'; arb^.stg := arb2; arb^.dr := arb1
    end;
    '^': begin
      Deriveaza(arbi^.stg, arb1); New(arb5); New(arb2);
      arb5^.info := '1'; arb5^.stg := nil; arb5^.dr := nil;
      arb2^.info := '-'; arb2^.stg := arbi^.dr; arb2^.dr := arb5;
      New(arb3); arb3^.info := '^'; arb3^.stg := arbi^.stg; arb3^.dr := arb2;
      New(arb4); arb4^.info := '*'; arb4^.stg := arbi^.dr; arb4^.dr := arb3;
      arb^.info := '*'; arb^.stg := arb4; arb^.dr := arb1
    end
  end
end;

```

```

{$S+}
{$S-}
procedure Tipareste(arb: arbore; nivel,x0: Integer);
const dy=30;
var i, dx: Integer;
begin
  dx := GetMaxX; for i := 1 to nivel do dx := dx div 2;
  if arb=nil then OutTextXY(x0+dx, nivel*dy, #207)
  else begin
    OutTextXY(x0+dx, nivel*dy, arb^.info);
    Line(x0+dx, nivel*dy+5, x0+dx div 2, (nivel+1)*dy-5);
    Line(x0+dx, nivel*dy+5, x0+dx+dx div 2, (nivel+1)*dy-5);
    Tipareste(arb^.stg,nivel+1,x0);Tipareste(arb^.dr,nivel+1,x0+dx)
  end
end;
{$S+}
{$S-}
procedure FormaPolonezaInversa(arb: arbore; var expr: expresie);
var expr1, expr2: expresie;
begin
  expr := '';
  if arb<>nil then
    begin
      FormaPolonezaInversa(arb^.stg, expr1);
      FormaPolonezaInversa(arb^.dr, expr2);
      expr := expr1 + ' ' + expr2 + ' ' +arb^.info
    end
  end;
{$S+}
begin { programul principal }
  ClrScr; WriteLn(' Dati expresia care va fi derivata ! ');
  ReadLn(expresia); expresia := expresia+' ';
  vectorizeaza(expresia,vectorul); New(arborele);
  Arborizeaza(vectorul,arborele);
  OpenGraph; SetTextJustify(CenterText, CenterText);
  OutTextXY(GetMaxX div 2, 10, Copy(expresia,1,Length(expresia)-1));
  FormaPolonezaInversa(arborele, expr_fpo);
  OutTextXY(GetMaxX div 2, GetMaxY-30, expr_fpo);
  Tipareste(arborele,1,0); Deriveaza(arborele, arborele_derivat);
  ReadLn; Dispose(arborele); ClearViewPort;
  FormaPolonezaInversa(arborele_derivat, expr_der_fpo);
  OutTextXY(GetMaxX div 2, GetMaxY-30, expr_der_fpo);
  Tipareste(arborele_derivat,1,0); ReadLn;
  Dispose(arborele_derivat); CloseGraph
end.

```

După cum se observă și din corpul programului principal anterior, pașii algoritmului sunt:

- * se citește expresia de derivat: *expresia*;
- * se memorează expresia într-un vector (*vector*), cu care se va lucra în continuare;
- * se construiește arborele asociat expresiei, din acest vector;
- * se derivează acest arbore, obținându-se arborele derivat al expresiei; derivarea se face cu o procedură recursivă, în care se studiază toate cazurile de rădăcină a arborelui în cauză.

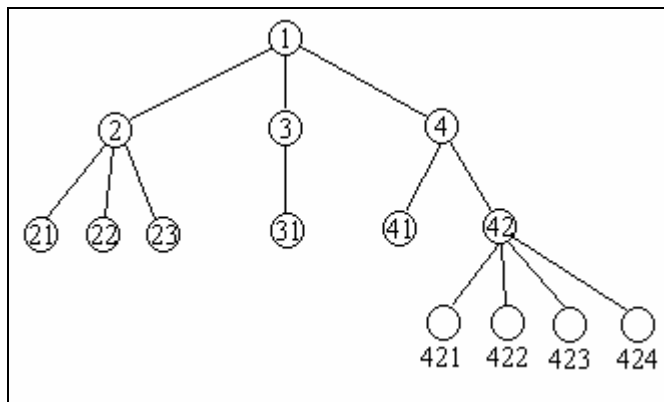
De fiecare dată se obține și forma postfixată (forma poloneză inversă) a arborelui (derivat sau nu), apoi fiecare din cei doi arbori se afișează sub formă grafică. De observat marea asemănare între procedura *arborizeaza* și funcția de evaluare a unei expresii algebrice, expusă în lecția 5, precum și naturalețea procedurii recursive de derivare *deriveaza*.

Lăsăm ca exercițiu cititorului obținerea expresiei derivate în forma normală, pornind de la arborele derivat.

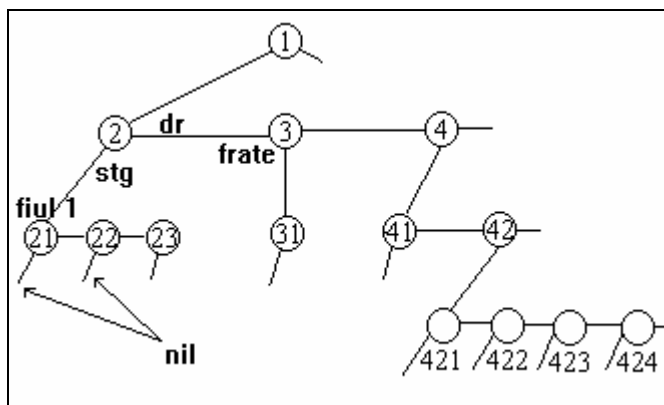
6. Memorarea arborilor oarecare în arbori binari

Prezentare teoretică

Într-adevăr, să considerăm un arbore oarecare, ca cel din figura următoare. Observăm că putem lega rădăcina 1 de nodul 2, iar apoi, nodul 2 poate fi legat de primul fiu al său (21) și de următorul fiu al rădăcinii, deci 3, despre care se spune că este un *frate* a lui 2. Procedând astfel pentru toate nodurile din arbore, vom obține un arbore binar, cu două legături: cea din stânga este către primul fiu, iar cea din dreapta către primul frate din dreapta al acestui fiu.



Astfel, arborele din figura anterioară se va memora în arborele binar de mai jos:



Arboreii oarecare îi vom memora sub forma unei structuri ce cuprinde o informație *info* (de tip *Integer*), un număr de fii (*NrFii*), precum și un vector de pointeri (referințe) către toți fiii nodului în cauză (*fiu: array[1..max] of arbore*). În cele ce urmează vom prezenta un program demonstrativ care va face următoarele:

- va citi un arbore oarecare A de la tastatură;
- va afișa acest arbore, pe verticală, așa cum este afișată structura de directoare DOS, în urma unei comenzi TREE;
- va memora arborele A sub forma unui arbore binar A2 (de tip arbore2);
- va afișa, în mod asemănător, acest arbore A2.

În privința procedurilor care execută aceste operații ale programului, trebuie să precizăm următoarele:

- ♦ procedura `Citeste` preia de la tastatură datele despre un arbore oarecare A în felul următor: se citește informația dintr-un nod (cel rădăcină), apoi numărul de fii pe care îi are acest nod; pentru fiecare fiu se va apela recursiv procedura, citindu-se subarborii care au ca rădăcini respectivii fii;
- ♦ procedura `TiparireÎnPreordine` (precum cazul ei particular dat de procedura `TiparireÎnPreordine2`) folosește caractere grafice speciale, pentru a desena structura de arbore. Aceste caractere grafice speciale formează diferite *antete*, care se afișează înaintea unui nod. (Aceste antete pot fi spații sau antete de verticale, care semnifică că suntem pe un nivel inferior, sau chiar antete depinzând de numărul de ordine al nodului, ca fiu al tatălui său. Steluța simbolizează nil.);
- ♦ procedura `TransfArb` este cea mai importantă, ea realizând transformarea unui arbore oarecare A într-un arbore binar A2; procedura este recursivă și se apelează pentru primul fiu al rădăcinii lui A (notat `fiu1`), precum și pentru toți ceilalți fii (între 2 și `NrFii`), care se înlanțuie la dreapta, în arborele A2.

Implementarea algoritmului

program TransformArbOarecareInArbBinar;

```
uses Crt;
const max=10;
type arbore = ^nod;
    nod=record NrFii:Byte; info:Integer; fiu: array[1..10] of arbore; end;
    arbore2 = ^nod2;
    nod2 = record info: Integer; stg,dr: arbore2 end;
```

```
procedure TransfArb(a: arbore; var a2: arbore2);
var fiu1:arbore; c,b,d: arbore2; i: Byte;
begin
    if a<>nil then
        begin
            New(a2); a2^.info:=a^.info; a2^.stg:=nil; a2^.dr:=nil;
            if a^.NrFii>0 then
                begin
                    fiu1:=a^.fiu[1]; TransfArb(fiu1,b); a2^.stg:=b; d:=b;
                    for i:=2 to a^.NrFii do
                        begin TransfArb(a^.fiu[i],c); b^.dr:=c; b:=b^.dr end
                    end
                end
            else a2:=nil
            end;
end;
```

```
procedure TiparireInPreordine2(A: arbore2);
const stnga=#195#196#196; dreapta=#192#196#196; vertical=#179#32#32;
    MaxNivele=20;
var antet: array[1..MaxNivele] of String; k: Integer;
procedure TPreordine(curent: arbore2);
var i: Integer;
```

```

begin
  for i:=0 to k do Write(antet[i]);
  if curent<>nil then WriteLn(curent^.info)
  else WriteLn('*');
  if curent<>nil then
    if (curent^.stg<>nil) or (curent^.dr<>nil) then
      if k<MaxNivele then
        begin
          if antet[k]=stinga then antet[k]:=vertical
          else antet[k]:='  ';
          k:=k+1; antet[k]:=stinga; TPreordine(curent^.stg);
          antet[k]:=dreapta; TPreordine(curent^.dr); k:=k-1
        end
      end;
begin k:=0; antet[k]:='-->'; TPreordine(A); end;
procedure Citeste(var A: arbore; nr, parinte: Integer);
var i: Byte;
begin
  New(A); Write('Dati info pt. fiul ',nr,' al lui ', parinte,' : ');
  ReadLn(A^.info);
  if A^.info<>0 then
    begin
      Write('Dati numarul de fii', 'pentru nodul ',A^.info,' : ');
      ReadLn(A^.NrFii);
      for i:=1 to A^.NrFii do Citeste(A^.fiu[i],i,A^.info);
    end
  else A:=nil
end;
procedure TiparireInPreordine(A: arbore);
const stinga=#195#196#196; dreapta=#192#196#196; vertical=#179#32#32;
  MaxNivele=20;
var antet: array[0..MaxNivele] of String[3]; k: Integer;
  procedure TPreordine(curent: arbore);
  var j,i: Integer;
  begin
    for i:=0 to k do Write(antet[i]);
    if curent<>nil then WriteLn(curent^.info)
    else WriteLn('*');
    if curent<>nil then
      if curent^.NrFii>0 then
        if k<MaxNivele then
          begin
            if antet[k]=stinga then antet[k]:=vertical
            else antet[k]:='  ';
            k:=k+1;
            for j:=1 to curent^.NrFii-1 do
              begin
                antet[k]:=stinga;
                TPreordine(curent^.fiu[j])
              end;
            antet[k]:=dreapta;
            with curent^ do TPreordine(fiu[NrFii]);
            k:=k-1
          end
        end;
    end;
begin k:=0; antet[k]:='-->'; TPreordine(A); end;
var A: arbore; A2: arbore2;
begin
  ClrScr; Citeste(A,0,0); TiparireInPreordine(A);
  TransfArb(A,A2); TiparireInPreordine2(A2); ReadLn
end.

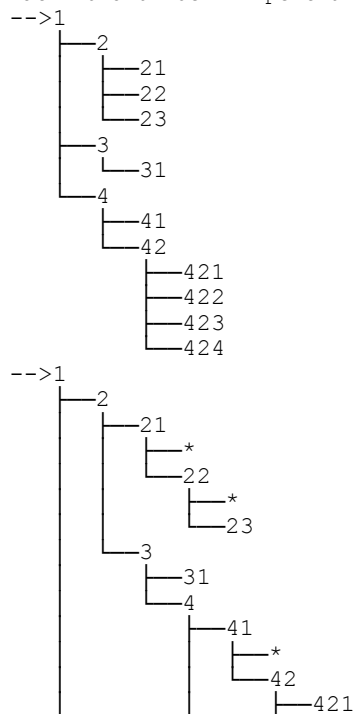
```

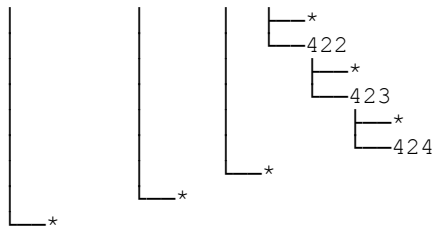

Iată un exemplu de funcționare a programului anterior pentru arborele oarecare considerat în figura dată în prezentarea teoretică:

```

Dati info pt. fiul 0 al lui 0: 1
Dati numarul de fii pentru nodul 1 : 3
Dati info pt. fiul 1 al lui 1: 2
Dati numarul de fii pentru nodul 2 : 3
Dati info pt. fiul 1 al lui 2: 21
Dati numarul de fii pentru nodul 21 : 0
Dati info pt. fiul 2 al lui 2: 22
Dati numarul de fii pentru nodul 22 : 0
Dati info pt. fiul 3 al lui 2: 23
Dati numarul de fii pentru nodul 23 : 0
Dati info pt. fiul 2 al lui 1: 3
Dati numarul de fii pentru nodul 3 : 1
Dati info pt. fiul 1 al lui 3: 31
Dati numarul de fii pentru nodul 31 : 0
Dati info pt. fiul 3 al lui 1: 4
Dati numarul de fii pentru nodul 4 : 2
Dati info pt. fiul 1 al lui 4: 41
Dati numarul de fii pentru nodul 41 : 0
Dati info pt. fiul 2 al lui 4: 42
Dati numarul de fii pentru nodul 42 : 4
Dati info pt. fiul 1 al lui 42: 421
Dati numarul de fii pentru nodul 421 : 0
Dati info pt. fiul 2 al lui 42: 422
Dati numarul de fii pentru nodul 422 : 0
Dati info pt. fiul 3 al lui 42: 423
Dati numarul de fii pentru nodul 423 : 0
Dati info pt. fiul 4 al lui 42: 424
Dati numarul de fii pentru nodul 424 : 0

```





7. Exerciții recapitulative

1. Se citesc numere întregi de la tastatură, până la întâlnirea numărului 0. Se cere să se creeze două liste, una a numerelor negative, iar alta a numerelor pozitive prime.
2. Se dă o listă de numere întregi pozitive. Să se creeze o listă care să conțină doar numerele pare, apoi să se concateneze cele două liste.
3. Se dau doi arbori binari. Se cere să se înlocuiască fiecare nod al primului cu cel de al doilea arbore.
4. Se dă un arbore oarecare, informațiile din noduri fiind șiruri de caractere. Să se construiască o listă dublu înlănțuită care să conțină toate șirurile din nodurile arborilor, care au lungimile pare, apoi să se ordoneze această listă.